



NATIONAL UNIVERSITY
of Computer & Emerging Sciences

Project Proposal:

TENSOR SCHED

Course: Operating System

Instructor: Miss Anum

Semester: Spring 2026

Group Members:

Rameen Ramzan (24K-0557)

Umama Zubair (24K-0621)

Introduction

Tensor Sched is a system designed to simulate a job scheduling environment where multiple jobs are submitted to a server at the same time. Each job requires CPU and system resources to execute. The system manages these jobs and schedules them efficiently using operating system concepts such as multithreading and synchronization.

The purpose of this project is to demonstrate how multiple processes can run concurrently while sharing limited resources. Tensor Sched provides a simplified environment to understand how operating systems handle job scheduling and prevent conflicts when multiple threads access shared data.

Problem Statement

In many computing systems, several jobs compete for limited resources such as CPU time and memory. Without proper scheduling and synchronization, problems such as resource conflicts, inefficient execution, and delays may occur.

This project addresses this problem by implementing a scheduling system that controls job execution and resource allocation. The system ensures that jobs are executed safely and efficiently using synchronization mechanisms.

Objectives

The objectives of this project are:

- To simulate a system where multiple jobs are submitted and executed concurrently.
- To demonstrate the concept of multithreading in operating systems.
- To manage shared resources using synchronization techniques.
- To implement job scheduling using a shared job queue.
- To observe how synchronization helps prevent resource conflicts.

OS Concepts Used

This project demonstrates several important operating system concepts:

Multithreading

Multiple threads will simulate job submissions and execution.

Shared Resource Table

The system maintains a shared structure that tracks available system resources.

Mutex Locks

Mutex locks are used to protect shared data such as job queues and resource tables from race conditions.

Semaphores

Semaphores control the number of jobs that can execute concurrently.

Job Scheduling

Jobs are scheduled based on resource availability and priority levels.

Methodology

The system follows these steps to manage job scheduling:

1. Users submit jobs to the system.
2. Each job is added to a shared job queue.
3. The scheduler checks available system resources.
4. Mutex locks ensure safe access to the job queue and resource table.
5. Semaphores control the number of jobs executing simultaneously.
6. Jobs are executed according to their priority and resource availability.
7. After execution, system resources are released and the next job is scheduled.

Technology Stack

- **Programming Language:** C++
- **Synchronization Tools:** Mutexes and Semaphones
- **Threading Model:** POSIX Threads (pthreads)
- **Interface:** Graphical User Interface

Final Product

The final product will be a job scheduling simulation system that demonstrates how operating systems manage concurrent processes and shared resources. The project will highlight the practical use of multithreading and synchronization techniques to ensure safe and efficient job execution.